

Y E H I O R T E C H

MédiHelm

SCAN CODE-BARRES MÉDICAMENTS — SPECS v1.0

Spécifications Techniques — Backend · Frontend · Prisma · API

Référence

MH-SCAN-SPECS-2025-v1.0

Périmètre

Scan EAN-13 · DataMatrix GS1 · Caméra PWA · Lecteur USB

Modules

M01 Stock · M02 POS · M01 Inventaire · M18 Vérification DPMED

Priorité

HAUTE — À implémenter après backend 16 modules

Dépendances

npm: @zxing/browser @zxing/library

Responsable

Dawes — Fondateur & Lead Developer, YEHI OR Tech

Document confidentiel — YEHI OR Tech © 2025

Sommaire

1. Architecture générale du scan
2. Parser GS1 — côté client
3. Composant BarcodeScanner (caméra PWA)
4. Hook useBarcodeInput (lecteur USB)
5. Endpoint POST /scan — NestJS
6. Service ScanResolverService — logique de résolution
7. Intégration POS — M02
8. Intégration réception stock — M01
9. Intégration inventaire — M01
10. Intégration vérification DPMED
11. Migrations Prisma — champs code-barres
12. Mode offline — SQLite
13. Index PostgreSQL — performances

1. Architecture Générale du Scan

```
// Flux complet d'un scan

[Lecteur USB / Caméra]
  |
  ▼
[parseGS1(rawCode)]      ← côté CLIENT - instantané
Extrait : gtin, lot, expiry, serie
  |
  ▼
POST /v1/scan            ← API NestJS
{ code, contexte, pharmacieId }
  |
  ▼
[ScanResolverService]
1. Chercher par EAN-13 / GTIN dans catalogue officine
2. Si non trouvé → base DCI nationale (DPMED)
3. Vérifier alertes DPMED sur le lot
4. Action selon contexte (POS / RECEPTION / INVENTAIRE / VERIF)
  |
  ▼
{ statut, médicament, lot, action, alerte? }
  |
  ▼
[Frontend] — mise à jour UI selon action
```

2. Parser GS1 — Côté Client

Le parser GS1 s'exécute côté client (frontend TypeScript) dès la réception du code brut — avant l'appel API. Il permet au frontend d'afficher immédiatement les informations disponibles (lot, expiration) sans attendre la réponse serveur.

```
// utils/gslParser.ts
export interface GS1Data {
  gtin?      : string; // GTIN-14 - AI 01
  ean13?     : string; // EAN-13 déduit du GTIN
  lot?       : string; // Numéro de lot - AI 10
  expiry?    : string; // YYMMDD brut - AI 17
  expiryDate?: Date;   // Date parsée
  serial?    : string; // Numéro de série - AI 21
  productionDate?: string;
  rawCode?   : string; // Code brut original
}

export function parseGS1(raw: string): GS1Data {
  const result: GS1Data = { rawCode: raw };

  // — Cas 1 : EAN-13 simple (13 chiffres purs) —————
  if (/^\d{13}$/.test(raw)) {
    result.ean13 = raw;
    result.gtin  = '0' + raw; // EAN-13 → GTIN-14
    return result;
  }

  // — Cas 2 : DataMatrix GS1 avec parenthèses (01)xxx(10)yyy —
  const withParens = [...raw.matchAll(/\((\d{2,4})\)([^\(]*)/g)];
  if (withParens.length > 0) {
    return extractAI(withParens, result);
  }

  // — Cas 3 : GS1-128 compact sans parenthèses —————
  // Commence par FNC1 (\x1d) ou directement par AI connu
  const compact = raw.replace(/^\x1d/, '');
  if (/^01\d{14}$/.test(compact)) {
    const pairs = compact.matchAll(/(\d{2,4})(.*?)?(?=\d{2,4}[^\\d]|$)/g);
    // Parser simplifié - longueurs fixes selon le standard GS1
    result.gtin  = compact.slice(2, 16);
    result.ean13 = result.gtin.slice(1);
    if (compact.includes('10')) {
      const idx = compact.indexOf('10', 16);
      if (idx !== -1) result.lot = compact.slice(idx+2, idx+22).trim();
    }
    if (compact.includes('17')) {
      const idx = compact.indexOf('17');
      if (idx !== -1) result.expiry = compact.slice(idx+2, idx+8);
    }
    return formatDates(result);
  }

  // — Cas 4 : Code non GS1 (Code 39, Code 128 interne) —————
  result.rawCode = raw; // Renvoyer tel quel - résolution côté serveur
}
```

```
    return result;
}

function extractAI(matches: RegExpMatchArray[], r: GS1Data): GS1Data {
  for (const [, ai, value] of matches) {
    const v = value.trim();
    if (ai === '01') { r.gtin = v; r.ean13 = v.slice(1); }
    if (ai === '10') r.lot = v;
    if (ai === '17') r.expiry = v;
    if (ai === '21') r.serial = v;
    if (ai === '11') r.productionDate = v;
  }
  return formatDates(r);
}

function formatDates(r: GS1Data): GS1Data {
  if (r.expiry?.length === 6) {
    const y = '20' + r.expiry.slice(0, 2);
    const m = r.expiry.slice(2, 4);
    // AI 17 : jour '00' = dernier jour du mois
    const d = r.expiry.slice(4, 6) === '00'
      ? new Date(+y, +m, 0).getDate().toString().padStart(2, '0')
      : r.expiry.slice(4, 6);
    r.expiryDate = new Date(`${y}-${m}-${d}`);
  }
  return r;
}
```

3. Composant BarcodeScanner (Caméra PWA)

```
// components/scan/BarcodeScanner.tsx
// Installation : npm install @zxing/browser @zxing/library
import { useEffect, useRef, useCallback } from 'react';
import { BrowserMultiFormatReader, NotFoundException } from '@zxing/browser';

interface Props {
  onScan      : (raw: string) => void;
  onError?    : (err: Error) => void;
  active      : boolean; // contrôlé par le parent
}

export function BarcodeScanner({ onScan, onError, active }: Props) {
  const videoRef = useRef<HTMLVideoElement>(null);
  const readerRef = useRef<BrowserMultiFormatReader | null>(null);

  const startScan = useCallback(async () => {
    if (!videoRef.current) return;
    const reader = new BrowserMultiFormatReader();
    readerRef.current = reader;
    try {
      await reader.decodeFromConstraints(
        { video: { facingMode: 'environment' } }, // caméra arrière
        videoRef.current,
        (result, error) => {
          if (result) {
            onScan(result.getText());
            reader.reset(); // un scan = stop
          }
          if (error && !(error instanceof NotFoundException)) {
            onError?.(error as Error);
          }
        }
      );
    } catch (e) {
      // Permission caméra refusée
      onError?.(e as Error);
    }
  }, [onScan, onError]);

  useEffect(() => {
    if (active) startScan();
    else readerRef.current?.reset();
    return () => { readerRef.current?.reset(); };
  }, [active, startScan]);

  return (
    <div className='relative w-full max-w-sm mx-auto'>
      <video
        ref={videoRef}
        className='w-full rounded-lg'
        style={{ display: active ? 'block' : 'none' }}
      />
      {/* Cadre de visée teal */}
      {active && (
```

```
        <div className='absolute inset-0 flex items-center justify-center'>
          <div className='w-48 h-32 border-2 border-teal-500 rounded opacity-
70' />
        </div>
      )}
    </div>
  );
}
```

4. Hook useBarcodeInput (Lecteur USB)

```
// hooks/useBarcodeInput.ts
// Le lecteur USB se comporte comme un clavier ultra-rapide.
// Caractères arrivant en < 50ms = scan. Entrée (\n) = fin du code.
import { useEffect, useRef, useCallback } from 'react';

interface Options {
  onBarcode      : (code: string) => void;
  minLength?    : number;    // défaut : 6
  speedThreshold? : number;   // ms entre chars, défaut : 50
  enabled?      : boolean;   // désactivable – défaut : true
}

export function useBarcodeInput({
  onBarcode,
  minLength      = 6,
  speedThreshold = 50,
  enabled        = true,
}: Options) {
  const bufferRef      = useRef('');
  const lastKeyTimeRef = useRef(0);

  const handleKeyDown = useCallback((e: KeyboardEvent) => {
    if (!enabled) return;

    const now      = Date.now();
    const elapsed = now - lastKeyTimeRef.current;
    lastKeyTimeRef.current = now;

    // Pause > 300ms = début de saisie humaine → vider le buffer
    if (elapsed > 300) bufferRef.current = '';

    if (e.key === 'Enter') {
      const code = bufferRef.current.trim();
      bufferRef.current = '';
      if (code.length >= minLength) {
        e.preventDefault(); // empêcher soumission de formulaire
        onBarcode(code);
      }
      return;
    }

    // Accumuler seulement si vitesse de frappe rapide (lecteur USB)
    if (elapsed < speedThreshold && e.key.length === 1) {
      bufferRef.current += e.key;
    } else if (e.key.length === 1) {
      // Frappe lente (humain) – reset buffer + premier caractère
      bufferRef.current = e.key;
    }
  }, [enabled, minLength, speedThreshold, onBarcode]);

  useEffect(() => {
    window.addEventListener('keydown', handleKeyDown);
    return () => window.removeEventListener('keydown', handleKeyDown);
  });
}
```

```
    }, [handleKeyDown]);  
  }
```

5. Endpoint POST /scan — NestJS

```
// scan/scan.controller.ts
@Controller('scan')
@UseGuards(JwtAuthGuard)
export class ScanController {
  constructor(private readonly scanService: ScanResolverService) {}

  @Post()
  @HttpCode(200)
  async scan(
    @Body() dto: ScanDto,
    @CurrentUser() user: JwtPayload,
  ) {
    return this.scanService.resoudre(dto.code, user.pharmacieId, dto.contexte);
  }
}

// scan/scan.dto.ts
export class ScanDto {
  @IsString() @NotEmpty()
  code: string;

  @IsEnum(ContexteScan)
  contexte: ContexteScan;
}

export enum ContexteScan {
  POS_VENTE = 'POS_VENTE',
  RECEPTION = 'RECEPTION',
  INVENTAIRE = 'INVENTAIRE',
  VERIFICATION = 'VERIFICATION',
}

// Réponses possibles
// { statut: 'TROUVE',      medicament, lot?, action }
// { statut: 'ALERTE',      medicament, lot, alerte: AlerteDPMED }
// { statut: 'NOUVEAU',     gslData, suggererAjout: true }
// { statut: 'INCONNU',     rawCode }
// { statut: 'PERIME',      medicament, lot } → bloquant au POS
// { statut: 'RUPTURE',     medicament }
```

6. Service ScanResolverService — Logique de Résolution

```
// scan/scan-resolver.service.ts
@Injectable()
export class ScanResolverService {
  constructor(private readonly prisma: PrismaService) {}

  async resoudre(rawCode: string, pharmacieId: string, contexte: ContexteScan)
  {
    const gs1 = parseGS1(rawCode); // parser GS1 - même logique que frontend

    // — Étape 1 : Recherche dans le catalogue de l'officine —
    const médicament = await this.chercherMédicament(gs1, pharmacieId);

    if (!médicament) {
      // — Étape 2 : Base DCI nationale (DPMED) —
      const national = await this.chercherBaseNationale(gs1);
      if (national) return { statut: 'NOUVEAU', médicament: national,
        gs1Data: gs1, suggererAjout: true };
      return { statut: 'INCONNU', rawCode };
    }

    // — Étape 3 : Vérification alerte DPMED sur le lot —
    if (gs1.lot) {
      const alerte = await this.verifierAlerteDPMED(gs1.lot);
      if (alerte) {
        return { statut: 'ALERTE', médicament, lot: gs1.lot, alerte };
      }
    }

    // — Étape 4 : Action selon contexte —
    switch (contexte) {

      case ContexteScan.POS_VENTE: {
        const lot = await this.trouverLotFEFO(médicament.id, pharmacieId);
        if (!lot) return { statut: 'RUPTURE', médicament };
        if (this.estPerime(lot)) return { statut: 'PERIME', médicament, lot };
        return { statut: 'TROUVE', médicament, lot, action: 'AJOUTER_PANIER' };
      }

      case ContexteScan.RECEPTION: {
        return {
          statut : 'TROUVE',
          médicament,
          gs1Data : gs1, // lot + expiry pré-remplis
          action : 'PREFILL_RECEPTION',
        };
      }

      case ContexteScan.INVENTAIRE: {
        const lot = gs1.lot
        ? await this.trouverLotParNumero(gs1.lot, médicament.id)
        : await this.trouverLotFEFO(médicament.id, pharmacieId);
      }
    }
  }
}
```

```

        return { statut: 'TROUVE', médicament, lot, action:
'SAISIR_QUANTITE' };
    }

    case ContexteScan.VERIFICATION: {
        const statutDPMED = await this.interrogerDPMED(gsl);
        return { statut: 'TROUVE', médicament, gslData: gsl, statutDPMED };
    }
}

}

// — Helpers —————
private async chercherMédicament(gsl: GS1Data, pharmacieId: string) {
    return this.prisma.medicament.findFirst({
        where: {
            pharmacieId,
            OR: [
                { codeBarres: gsl.ean13 },
                { gtin:      gsl.gtin },
                { codeBarres: gsl.rawCode },
            ],
            actif: true,
        },
    });
}

private async trouverLotFEFO(medicamentId: string, pharmacieId: string) {
    return this.prisma.lot.findFirst({
        where: {
            médicamentId,
            médicament: { pharmacieId },
            quantite:    { gt: 0 },
            dateExpiration: { gt: new Date() },
        },
        orderBy: { dateExpiration: 'asc' }, // FEFO
    });
}

private estPerime(lot: { dateExpiration: Date }): boolean {
    return lot.dateExpiration < new Date();
}

private async verifierAlerteDPMED(numeroLot: string) {
    return this.prisma.alerteDPMED.findFirst({
        where: {
            numerosLotConcernes: { has: numeroLot },
            statut: 'EN_DIFFUSION',
        },
    });
}
}

```

7. Intégration POS — M02

```
// components/pos/POSPage.tsx — intégration scan

export function POSPage() {
  const [scannerActif, setScannerActif] = useState(false);
  const { ajouterAuPanier, setErreur } = usePanier();

  // Hook lecteur USB — actif en permanence sur la page POS
  useBarcodeInput({
    onBarcode: (code) => handleScan(code),
    enabled: true,
  });

  const handleScan = async (rawCode: string) => {
    const gsl = parseGSl(rawCode); // parsing immédiat côté client

    const result = await fetch('/api/scan', {
      method: 'POST',
      body: JSON.stringify({ code: rawCode, contexte: 'POS_VENTE' }),
    }).then(r => r.json());

    switch (result.statut) {
      case 'TROUVE':
        ajouterAuPanier(result.medicament, result.lot);
        break;
      case 'ALERTE':
        // Afficher modal alerte DPMED — bloquant si URGENCE_IMMEDIATE
        afficherAlerteModal(result.alerte, result.medicament);
        break;
      case 'PERIME':
        setErreur(`Ce lot est périmé depuis le
        ${format(result.lot.dateExpiration)}`);
        break;
      case 'RUPTURE':
        setErreur(`Rupture de stock — ${result.medicament.nomCommercial}`);
        break;
      case 'NOUVEAU':
        afficherPropositionAjout(result.gslData);
        break;
      case 'INCONNU':
        setErreur('Code non reconnu — saisie manuelle');
        break;
    }
  };

  return (
    <div>
      { /* Bouton caméra — alternatif au lecteur USB */ }
      <button onClick={() => setScannerActif(true)}>
        □ Scanner
      </button>
      <BarcodeScanner
        active={scannerActif}
        onScan={(code) => { setScannerActif(false); handleScan(code); }}
        onError={() => setScannerActif(false)}
      />
    </div>
  );
}
```

```
    />  
    { /* ... reste du POS */ }  
  </div>  
);  
}
```

8. Intégration Réception Stock — M01

```
// components/stock/ReceptionForm.tsx

export function ReceptionForm() {
  const [scannerActif, setScannerActif] = useState(false);
  const [formData, setFormData] = useState({
    medicamentId: '', gtin: '', numeroLot: '', dateExpiration: '',
    quantite: 0, prixAchat: 0,
  });

  // Lecteur USB actif sur cet écran
  useBarcodeInput({ onBarcode: handleScan });

  const handleScan = async (rawCode: string) => {
    const result = await fetch('/api/scan', {
      method: 'POST',
      body: JSON.stringify({ code: rawCode, contexte: 'RECEPTION' }),
    }).then(r => r.json());

    if (result.statut === 'TROUVE' || result.statut === 'NOUVEAU') {
      // Pré-remplissage automatique du formulaire
      setFormData(prev => ({
        ...prev,
        medicamentId : result.medicament?.id ?? '',
        gtin           : result.gslData?.gtin ?? '',
        numeroLot      : result.gslData?.lot ?? '',
        dateExpiration: result.gslData?.expiry ?? '',
        // quantite et prixAchat restent à saisir manuellement
      }));
    }
    if (result.statut === 'NOUVEAU') {
      // Proposer création du médicament dans le catalogue
      setPropositionAjout(result.gslData);
    }
  };

  return (
    <form>
      <button type='button' onClick={() => setScannerActif(true)}>
        ☐ Scanner la boîte reçue
      </button>
      <BarcodeScanner active={scannerActif} onScan={handleScan} />
      <input value={formData.numeroLot}
        onChange={e => setFormData(p => ({...p, numeroLot:
e.target.value}))}
        placeholder='Numéro de lot' />
      <input value={formData.dateExpiration}
        placeholder='Date expiration' />
      <input type='number' placeholder='Quantité reçue' />
      <input type='number' placeholder='Prix achat (FCFA)' />
    </form>
  );
}
```

9. Intégration Inventaire — M01

```
// components/stock/InventaireSession.tsx

export function InventaireSession({ sessionId }: { sessionId: string }) {
  const [scanResult, setScanResult] = useState<ScanResult | null>(null);
  const [quantiteReelle, setQuantiteReelle] = useState('');

  useBarcodeInput({
    onBarcode: async (rawCode) => {
      const result = await fetch('/api/scan', {
        method: 'POST',
        body: JSON.stringify({ code: rawCode, contexte: 'INVENTAIRE' }),
      }).then(r => r.json());

      setScanResult(result);
      setQuantiteReelle(''); // reset pour ce lot
      // Focus automatique sur le champ quantité
      document.getElementById('quantite-input')?.focus();
    }
  });

  const enregistrerQuantite = async () => {
    await
    fetch(`/api/stock/inventaire/${sessionId}/lots/${scanResult!.lot.id}`, {
      method: 'PATCH',
      body: JSON.stringify({ quantiteReelle: +quantiteReelle }),
    });
    setScanResult(null); // prêt pour le prochain scan
  };

  return (
    <div>
      {scanResult ? (
        <div>
          <p>{scanResult.medicament.nomCommercial}</p>
          <p>Lot : {scanResult.lot.numeroLot}</p>
          <p>Stock système : {scanResult.lot.quantite}</p>
          <input id='quantite-input'
            type='number'
            value={quantiteReelle}
            onChange={e => setQuantiteReelle(e.target.value)}
            placeholder='Quantité comptée'
          />
          <button onClick={enregistrerQuantite}>Valider</button>
        </div>
      ) : (
        <p>☐ Scannez une boîte pour saisir la quantité réelle</p>
      )}
    </div>
  );
}
```

11. Migrations Prisma — Champs Code-barres

```
// prisma/migrations/YYYYMMDD_add_scan_fields/migration.sql

-- Patch table médicaments
ALTER TABLE médicaments
  ADD COLUMN IF NOT EXISTS gtin          VARCHAR(14),
  ADD COLUMN IF NOT EXISTS code_amm     VARCHAR(50);

-- Index composites pour résolution rapide au scan
CREATE INDEX IF NOT EXISTS idx_medicaments_codebarres
  ON médicaments(pharmacie_id, code_barres);

CREATE INDEX IF NOT EXISTS idx_medicaments_gtin
  ON médicaments(pharmacie_id, gtin);

-- Patch table lots
ALTER TABLE lots
  ADD COLUMN IF NOT EXISTS gtin_lot      VARCHAR(14),
  ADD COLUMN IF NOT EXISTS numero_serie VARCHAR(50),
  ADD COLUMN IF NOT EXISTS raw_barcode   TEXT;
```

```
// schema.prisma - ajouts sur les modèles existants

model Medicament {
  // ... champs existants ...
  codeBarres String? // EAN-13 - existant
  gtin       String? // GTIN-14 GS1 - NOUVEAU
  codeAMM    String? // AMM DPMED Bénin - NOUVEAU

  @@index([pharmacieId, codeBarres])
  @@index([pharmacieId, gtin])
}

model Lot {
  // ... champs existants ...
  gtinLot      String? // GTIN du lot - NOUVEAU
  numeroSerie  String? // Série GS1 AI-21 - NOUVEAU
  rawBarcode    String? // Code brut scanné - NOUVEAU
}
```

12. Mode Offline — SQLite

Le scan doit fonctionner en mode offline POS. Les médicaments et lots sont synchronisés dans SQLite au login (et toutes les 5 minutes si connecté). L'endpoint POST /scan a son équivalent local qui interroge SQLite directement — même logique de résolution.

```
// services/scan-offline.service.ts
// Même interface que ScanResolverService — mais interroge SQLite

export class ScanOfflineService {
  constructor(private readonly sqliteDb: SQLitePrismaClient) {}

  async resoudre(rawCode: string, pharmacieId: string, contexte: ContexteScan)
  {
    const gs1 = parseGS1(rawCode);
    const medicament = await this.sqliteDb.medicament.findFirst({
      where: {
        pharmacieId,
        OR: [{ codeBarres: gs1.ean13 }, { gtin: gs1.gtin }],
      },
    });
    if (!medicament) return { statut: 'INCONNU', rawCode };
    // Même logique FEFO et vérification péremption
    // Note : les alertes DPMED ne sont PAS vérifiables offline
    const lot = await this.trouverLotFEFO(medicament.id);
    if (!lot) return { statut: 'RUPTURE', medicament };
    return { statut: 'TROUVE', medicament, lot, action: 'AJOUTER_PANIER' };
  }
}

// Dans le composant POS — sélection du service selon l'état réseau
const scanService = navigator.onLine ? onlineScan : offlineScan;
```